

Speaker Notes: LLM-Guided Rule Discovery for Web Attack Detection

Rahul Tripathi, Armaan Oberai, Justine Ellery

Slide 1: Title slide

Timing: 15 seconds.

Open by saying this project asks whether an LLM can help write useful web-attack detection rules without being used as the detector. The key distinction is that the LLM proposes rules offline, but the final runtime detector is just deterministic regexes and endpoint heuristics.

Slide 2: This attack does not look like a classic injection

Timing: 55 seconds.

Start with the example request. Ask the audience what looks suspicious. Then reveal that the issue is not a classic injection token, but an endpoint invariant: price should be numeric. This sets up the whole project.

Slide 3: Accurate detectors are not always deployable rules

Timing: 45 seconds.

Now generalize from the hook. A classifier can be accurate, but operators often want concrete detection logic they can inspect and place into a rule-based enforcement pipeline.

Slide 4: Before LLMs, web attack detection split between rules and models

Timing: 45 seconds.

Keep this high level. The point is not to survey every paper. Before LLMs, there were two main tracks: hand-built rules that operators could deploy, and statistical or ML models that often performed well but were harder to inspect or translate into firewall logic.

Slide 5: LLMs changed rule generation, but often target heavy rule languages

Timing: 45 seconds.

This slide should bridge into our contribution. LLM rule-generation work is very relevant, but much of it asks the LLM to produce rules for Suricata, ModSecurity,

KQL, or similar systems. Those languages are useful, but they bring their own syntax, lifecycle, and validation complexity.

Slide 6: Our project studies a smaller, controlled form of rule discovery

Timing: 50 seconds.

This is the positioning slide. Our project is intentionally smaller than production rule-generation systems. That is the point: isolate the discovery loop, keep the output simple, and compare it fairly to a traditional machine-learning baseline.

Slide 7: CSIC 2010 web-application requests

Timing: 40 seconds.

Introduce CSIC 2010 as the benchmark. It is useful because it contains labeled normal and anomalous HTTP requests from a small web application. Mention that the dataset is structured, which is both helpful and a limitation. The reported results use an 80/20 split.

Slide 8: Use the LLM offline, not in the detector

Timing: 55 seconds.

This is the central slide. Walk through the loop: examples and errors go to the LLM, the LLM proposes candidates, the harness evaluates them, and only useful rules survive. The final output is frozen rules, not a prompt or a model. That is what makes runtime cheap and auditable.

Slide 9: Rule examples: combine attack syntax with endpoint structure

Timing: 50 seconds.

Explain that the early rules were obvious signatures, like SQL and XSS. Those had high precision but missed many attacks. The larger improvement came from endpoint-specific checks: values that should be numeric, fields that should not appear on a form, or parameters that should follow a narrow format. This is why the LLM loop is useful: it helps suggest where local invariants exist.

Slide 10: Example: A numeric-field invariant catches subtle mutations

Timing: 50 seconds.

Use this as a concrete example of the method. A generic attack regex might miss this because it does not contain an obvious injection token. But the endpoint has

a local grammar: price and quantity should look numeric. The LLM proposes that invariant, and the harness decides whether it improves metrics.

Slide 11: Accuracy improves across rule-discovery rounds

Timing: 50 seconds.

Walk through the curve. The first round catches obvious strings, but accuracy is low because many attacks are structural. As the loop adds endpoint schemas and value constraints, performance climbs. The final rule set reaches 99.21 percent test accuracy.

Slide 12: The frozen rule set reaches 99.21% held-out accuracy

Timing: 45 seconds.

This slide gives the result before the detailed comparison. Keep it simple: the rule set is accurate, precise, and evaluated on the held-out split. The next section will compare this directly against the reproduced random-forest baseline.

Slide 13: The baseline is a reproduced random-forest classifier

Timing: 40 seconds.

Set up the comparison before showing numbers. The baseline is not a strawman: it uses character TF-IDF over request text plus numeric features and trains a random forest. Emphasize that both systems are evaluated on the same 20 percent test split.

Slide 14: Traditional ML vs. LLM-guided rules: held-out accuracy and F1

Timing: 55 seconds.

This slide should feel like the main comparison figure. First show the reproduced ML baseline, then reveal the rule harness. Keep the caveat explicit: this is not a universal claim about all ML or all web traffic. It is a result on this benchmark and split.

Slide 15: The rule detector makes very few benign mistakes

Timing: 40 seconds.

Bring back the confusion matrix from the paper as part of the ML comparison section. The important point is not just high accuracy. It is that the rule set is very conservative on benign traffic, with only 4 false positives.

Slide 16: Runtime comparison: rules remove the heavy pipeline

Timing: 50 seconds.

Explain carefully that the runtime comparison is not pure model inference. The random forest timing includes the local notebook pipeline. Still, this slide shows why a frozen rule set is appealing for deployment: no model call, no training path, and cheap deterministic evaluation.

Slide 17: Limitations and next steps

Timing: 55 seconds.

Be candid here. The strongest limitation is that the rule families were developed through earlier dataset inspection, even though the final evaluation uses a held-out 20 percent split. The clean next experiment is to freeze the split before any rule discovery, let the agent see only training or development data, and then report once on the test set. Also mention that CSIC is synthetic and structured.

Slide 18: LLMs can help write rules without becoming detectors

Timing: 35 seconds.

End with the clean takeaway. The LLM is useful because it helps search the rule space, but the harness is what makes the system reliable. The final detector is deterministic and inspectable. That is the contribution of the project.

Slide 19: Summary

Timing: open-ended.

Use this as the final landing slide. Re-state the main result in one sentence, then leave the Q and A visible while taking questions.